

Zero-Day Behavior Anomaly Detection: Behavior-Based Anomaly Detection Engine

AWAB HAIDER ABDALLAH ABDELRAHMAN

Kastamonu University, Department of Computer Engineering, 2nd Year

Database Management Systems Course Project Report

Student ID: 234410188

Abstract

This report presents the Zero-Day Behavior Anomaly Detection system, a web application developed with ASP.NET Core MVC. The project implements anomaly detection by extracting user behavior profiles. The system learns normal behavior patterns of users (login hours, IP addresses, operation types, resource accesses) and flags activities that deviate from these patterns as zero-day anomalies. The application demonstrates in-memory data structures, behavior profiling, rule-based anomaly scoring, seed data, and dashboard statistics, making it suitable for a Database Management Systems course project.

Keywords: ASP.NET Core MVC, Anomaly Detection, Behavior Profile, Zero-Day, Rule-Based System, Database Management Systems.

I. Introduction

In cybersecurity, detection systems based on known attack signatures are often insufficient against new and unknown threats. These threats, called zero-day attacks, use previously unseen attack vectors. Therefore, anomaly detection systems based on user behavior have become increasingly important.

The Zero-Day Behavior Anomaly Detection system was developed as an academic project for a Database Management Systems course. The project analyzes users' normal activities on the system to build a profile and detects activities that do not match this profile in real time. This enables not only data storage and listing operations but also meaningful security analysis.

II. Purpose and Scope

The main purpose of the project is to develop an anomaly detection engine based on user behavior profiles and present it through a web-based interface.

The main goals of the project are:

- Learn normal behavior patterns of users (time, IP, operation, resource) and store them as profiles
- Compare new log entries with existing profiles and calculate anomaly scores
- Flag entries that exceed the anomaly score threshold and display warnings to the user
- Display total logs, anomaly count, anomaly rate, and severity statistics on the dashboard
- List detected anomaly reasons on the anomaly detail page
- Allow users to manually add logs and have them analyzed instantly

III. Technologies Used

The application is an ASP.NET Core MVC project targeting .NET 9. In-memory data structures (List, Dictionary) are

used for data storage. The user interface is built with Razor Views and Bootstrap 5. The project does not require an external database server; all data is kept in memory during application runtime.

Technology	Purpose
ASP.NET Core MVC	Web architecture based on controllers, models, and views

.NET 9

IV. System Architecture

The project follows the MVC architecture. Controller classes handle HTTP requests and use the Service layer. The anomaly detection logic is not written directly inside the controller; it is separated through the AnomalyDetectionService class. This structure centralizes the business rule and improves code readability.

Layer	Example Files	Responsibility
Models	Islem, Profil, DashboardViewModel	Data entities and view models

Service

V. Data Structure Design

The system is built around three main data structures: Islem (LogEntry), Profil (UserBehaviorProfile), and DashboardViewModel. The Islem class represents each log record and contains fields for islem_id, tarih, kul_id, kul_adi, islem, kaynak, ip, basarili, anormal, puan, and neden. The Profil class stores a user's normal behavior patterns and consists of lists for normal_saatler, normal_iplar, normal_islemler, and normal_kaynaklar.

Each user can have multiple log records. User profiles are extracted from normal (non-anomalous) log records. Anomaly detection is performed by comparing a new log entry against the relevant user's profile.

Class	Main Fields	Description
Islem	islem_id, tarih, kul_id, kul_adi, islem, kaynak, ip, basarili, anormal, puan, neden	Log record and anomaly information

Profil

VI. Data Integrity and Validation

The islem_id field is managed with an auto-incrementing counter, ensuring each record has a unique identifier. User profiles are stored in a Dictionary structure keyed by kul_id, preventing duplicate profiles for the same user. The anomaly score is calculated in the 0-100 range, and entries exceeding the threshold value (Threshold = 50) are flagged as anomalies. During profile building, only normal (non-anomalous) records are used, which increases profile reliability.

VII. Anomaly Detection Algorithm

The anomaly detection process is implemented in AnomalyDetectionService. When a new log record is created or analyzed, the service is called and compares the entry against the relevant user's profile.

The algorithm evaluates four main factors:

- Time mismatch: If the log hour is outside the user's normal hours, +30 points
- IP mismatch: If the IP address is outside the user's normal IPs, +25 points
- Operation mismatch: If the operation type is outside the user's normal operations, +25 points
- Resource mismatch: If the resource is outside the user's normal resources, +20 points

If the total score is 50 or above, the record is flagged as an anomaly.

Condition	Score Contribution
Unusual time	+30

Unknow

VIII. Application Modules

Dashboard Module:

Displays the total number of logs, detected anomaly count, anomaly rate, log count in the last 24 hours, and anomaly distribution by severity levels. Recently detected anomalies are also listed in a table.

Log Management Module:

Includes listing all log records, adding new logs, and viewing anomaly status. When a new log is added, the system automatically performs anomaly analysis and notifies the user of the result.

Anomaly Reports Module:

Lists all detected anomalies, and the detail page shows the anomaly score, severity level, and detected anomaly reasons.

Profile Management (background):

The BehaviorProfileService automatically creates and updates user profiles from normal log records.

IX. Seed Data

Sample data is generated through the SampleData.cs class. There are three different users (Ahmet Yılmaz, Zeynep Kaya, Mehmet Demir) with 5 days of normal activity records and 5 anomaly scenarios.

Normal records include Login, ViewFile, EditFile, ViewFile, and Logout operations for each user, occurring at specific time ranges from fixed IP addresses.

Anomaly scenarios include:

- Login from unknown IP (Ahmet Yılmaz - 185.220.101.42)
- Unusual operation and resource (Ahmet Yılmaz - DeleteFile, /critical/data.db)
- Unknown IP, operation, and resource (Zeynep Kaya - ExecuteCommand, /bin/sh, 91.234.56.78)
- Unknown IP and time (Mehmet Demir - 10.0.0.5)
- Unknown IP and resource (Zeynep Kaya - DownloadFile, /confidential/musteri_verileri.xlsx, 192.168.1.200)

This seed data makes the application testable immediately after startup.

X. Sample Scenario

When a user creates a new log entry for Ahmet Yilmaz, the system follows these steps:

1. The user enters "Login" as the operation and "185.220.101.42" as the IP address.
2. The log entry is created and AnomalyDetectionService is called.
3. The service loads Ahmet Yilmaz's profile. The profile contains only '192.168.1.100' as a normal IP.
4. The login time (e.g., 16:00) is outside Ahmet's normal hours (8-18), so +30 points are added.
5. The IP address is not found in the profile, so +25 points are added.
6. The total score is 55, which exceeds the threshold (50), so the record is flagged as an anomaly.
7. The user sees a warning: "ZERO-DAY ANOMALY DETECTED! Score: 55".
8. The record can be examined in detail on the Anomaly Reports page.

XI. Conclusion

The Zero-Day Behavior Anomaly Detection system is a practical Database Management Systems project that demonstrates behavior-based anomaly detection, profile extraction, rule-based scoring, and seed data operations. Developed with ASP.NET Core MVC, the system detects zero-day threats by extracting user behavior profiles. Through the rule-based anomaly detection engine, log records are not only stored but also transformed into meaningful security analyses. The project can be extended with machine learning-based anomaly detection, real-time alert systems, graphical reporting, and multi-database support.

References

- [1] Microsoft, "ASP.NET Core MVC documentation," Microsoft Learn.
- [2] Microsoft, "Building Web APIs with ASP.NET Core," Microsoft Learn.
- [3] Zero-Day Behavior Anomaly Detection project source code and README file.
- [4] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys.